EmotionAI – Multimodal Emotion Detection System

INDEX

Sr. No.	Topic	Page No.
1	Introduction	1
2	Requirement Analysis	2
3	Development Technologies	3
4	Modelling Classes	6
5	Progress Assessment 1 & 2 (Week 6 & 9)	10
6	Implementation and Output	12
7	Observations and Learnings	22
8	Future Scope	23
9	Conclusion	24
10	References	25

1. Introduction

Emotion recognition is one of the most significant challenges in Artificial Intelligence and Human-Computer Interaction (HCI). The ability of machines to perceive and understand human emotional states has wide-ranging applications: mental health monitoring, adaptive learning systems, driver safety alerts, customer sentiment analysis, and interactive entertainment.

EmotionAI is a full-stack machine learning web application that detects human emotions through two complementary modalities:

- Face-based Emotion Detection: Real-time analysis of facial expressions captured via a live webcam feed. The system classifies expressions into 7 emotion categories using a custom-built Convolutional Neural Network (CNN) trained on the FER-2013 dataset.
- Text-based Sentiment Analysis: Natural language input is analyzed using a fine-tuned DistilBERT transformer model, capable of detecting 28 distinct emotional states from user-provided text using the GoEmotions dataset.

The project bridges research-grade machine learning with real-world deployment, integrating a React-based frontend, a Flask REST API backend, and two independent ML inference services into a cohesive, interactive web application.

The system is designed with modularity in mind: the ML training code, backend API, and frontend UI are fully decoupled, making it easy to extend with new modalities or improved models in the future.

2. Requirement Analysis

2.1 Problem Statement

Traditional HCI systems are unable to perceive or respond to human emotional states. Existing solutions analyze either facial expressions or text in isolation, reducing both accuracy and contextual richness. EmotionAI addresses this gap by building a unified, multimodal system that simultaneously processes facial and textual inputs to deliver comprehensive emotional intelligence in real time.

2.2 Functional Requirements

- The system shall capture live webcam frames and classify facial expressions into one of 7 emotion categories: angry, disgust, fear, happy, neutral, sad, and surprise.
- The system shall accept natural language text and classify it into one or more of 28 emotion categories using multi-label classification.
- The system shall display emotion probability scores as sorted visual bar charts in the user interface.
- The backend shall expose REST API endpoints for face prediction (/api/face/predict) and text prediction (/api/text/predict).
- The system shall provide a /api/health endpoint to confirm backend availability.
- The frontend shall be a single-page React application supporting real-time webcam capture and text input.

2.3 Non-Functional Requirements

- Performance: Inference response time under 3 seconds for both modalities.
- Accuracy: Face CNN should achieve at least 60% on FER-2013; text model should achieve competitive subset accuracy for 28-class multi-label classification.
- Usability: Clean, dark-themed UI functional on all modern browsers with webcam support.
- Modularity: ML training, backend, and frontend components separated for independent development and testing.
- Scalability: REST API architecture supports future addition of new emotion modalities.

2.4 Hardware and Software Requirements

Category	Requirement
Operating System	Windows 10/11 (development) or Ubuntu Linux
Python Version	Python 3.10+
Node.js Version	Node.js 18+ with npm
GPU (Recommended)	NVIDIA CUDA-capable GPU for model training
RAM	Minimum 8 GB (16 GB recommended for training)
Storage	At least 10 GB free for datasets and model checkpoints
Browser	Chrome / Firefox with webcam support
Internet	Required for initial dataset and dependency downloads

3. Development Technologies

3.1 Technology Stack

Layer	Technology	Purpose
ML Training	Python + PyTorch	Train face CNN and DistilBERT text models
Backend API	Python + Flask	Serve model predictions via REST API
Frontend UI	React + Vite	User interface with webcam and text input
Face Detection	OpenCV (Haar Cascade)	Detect and crop face region from frames
Text Tokenization	HuggingFace Transformers	Tokenize text for DistilBERT model
HTTP Client	Axios	Frontend API calls to Flask backend
Charting	Recharts	Emotion probability bar visualization

3.2 Languages Used

- Python 3.10+ — ML training, backend API, data preprocessing
- JavaScript (React) — Frontend user interface
- CSS — Styling and layout

3.3 Deep Learning Packages

Package	Version	Used For	Location in Project
torch	Latest	Core deep learning framework, tensors, autograd	face_model/, text_model/, services/
torchvision	Latest	Image transforms, pretrained models, ImageFolder dataset	face_model/data_loader.py
transformers	Latest	DistilBERT model and tokenizer (HuggingFace)	text_model/model.py, text_sentiment_service.py
datasets	Latest	Load GoEmotions dataset from HuggingFace Hub	datasets/download_text_data.py
tokenizers	Latest	Fast tokenization backend for HuggingFace	text_model/tokenizer_utils.py

3.4 Computer Vision & Data Science Packages

Package	Used For	Location
opencv-python	Face detection (Haar Cascade), image decoding, grayscale conversion	face_emotion_service.py
albumentations	Advanced image augmentation during training	face_model/data_loader.py
Pillow	Image loading, format conversion	face_model/data_loader.py
numpy	Array operations, image pixel manipulation	face_emotion_service.py
pandas	CSV reading/writing, dataset manipulation	datasets/, text_model/
scikit-learn	Metrics: classification_report, accuracy_score, train/test splits	evaluate.py files

Package	Used For	Location
matplotlib	Plotting training loss/accuracy curves	train.py files
seaborn	Confusion matrix visualization	evaluate.py files

3.5 Backend Packages

Package	Used For	Location
flask	Web framework, REST API server	backend/app.py, routes/
flask-cors	Allow React frontend to call Flask API (cross-origin)	backend/app.py
gunicorn	Production WSGI server	Deployment
python-dotenv	Load environment variables from .env file	backend/config.py
kaggle	Download FER-2013 dataset from Kaggle CLI	Dataset download

3.6 Frontend (JavaScript) Packages

Package	Used For	Location
react	UI component framework	All frontend components
react-dom	Render React components to browser DOM	frontend/src/main.jsx
vite	Fast dev server and build tool	frontend/vite.config.js
axios	HTTP client for calling Flask backend API	frontend/src/api/emotionApi.js
react-webcam	Webcam access and screenshot capture in React	frontend/src/components/FaceAnalysis.jsx
recharts	Chart library for emotion probability bars	frontend/src/components/

4. Modelling Classes

4.1 Face Emotion Model – Custom CNN

Type: Convolutional Neural Network (CNN) built from scratch using PyTorch.

Dataset: FER-2013 (Facial Expression Recognition 2013) — 28,709 training images, 7,178 test images, 48×48 pixel grayscale, 7 emotion classes.

CNN Architecture (EmotionCNN):

Layer	Details	Output Shape
Input	Grayscale image	1 × 48 × 48
Block 1	Conv2d(1→32) + BatchNorm + ReLU + MaxPool + Dropout	32 × 24 × 24
Block 2	Conv2d(32→64) + BatchNorm + ReLU + MaxPool + Dropout	64 × 12 × 12
Block 3	Conv2d(64→128) + BatchNorm + ReLU + MaxPool + Dropout	128 × 6 × 6
Flatten	Reshape to 1D vector	4608
FC1	Linear(4608→256) + ReLU + BatchNorm + Dropout(0.5)	256
FC2	Linear(256→128) + ReLU + Dropout(0.3)	128
Output	Linear(128→7) — 7 emotion class logits	7

Training Configuration:

- Epochs: 50
- Optimizer: Adam
- Loss Function: CrossEntropyLoss with class weights (to handle class imbalance — disgust: 111 samples vs. happy: 7,215 samples)
- LR Scheduler: ReduceLROnPlateau
- Augmentation: Random horizontal flip, random rotation (±10°), color jitter
- Final Test Accuracy: 65% on FER-2013 test set

4.2 Text Sentiment Model – DistilBERT

Type: Fine-tuned Transformer (DistilBERT) using HuggingFace Transformers.

Dataset: GoEmotions (Google) — 43,410 training / 5,426 validation / 5,427 test samples of Reddit comments labeled with 28 emotion categories. Multi-label: one text can have multiple emotions.

Model Architecture (TextEmotionModel):

Layer	Details
Input	Tokenized text (max 128 tokens, padded/truncated)
DistilBERT Base	Pretrained on Wikipedia + BookCorpus (66M parameters)
CLS Token	Extract [CLS] representation — 768 dimensions
FC1	Linear(768→256) + ReLU + BatchNorm + Dropout(0.3)
FC2	Linear(256→128) + ReLU + Dropout(0.3)
Output	Linear(128→28) + Sigmoid (multi-label, one score per emotion)

Training Configuration:

- Epochs: 10
- Optimizer: AdamW with weight decay
- Loss Function: BCEWithLogitsLoss (Binary Cross Entropy for multi-label)
- LR Scheduler: StepLR
- Gradient Clipping: Applied to prevent exploding gradients
- Inference Threshold: 0.3 on sigmoid outputs
- Final Subset Accuracy: 42.78% (strong for 28-class multi-label classification)

4.3 Face Detection – Haar Cascade

Type: Classical Computer Vision (not deep learning). Library: OpenCV built-in haarcascade_frontalface_default.xml.

Purpose: Detect and crop the face region from webcam frames before passing to the CNN. Uses a sliding window with Haar-like features and AdaBoost ensemble classifier. If no face is detected, the full image is used as fallback.

4.4 FER-2013 Emotion Classes (7)

angry, disgust, fear, happy, neutral, sad, surprise

Note: 'disgust' has only 111 training samples vs. 'happy' with 7,215 — severe class imbalance handled with weighted CrossEntropyLoss.

4.5 GoEmotions Emotion Classes (28)

admiration, amusement, anger, annoyance, approval, caring, confusion, curiosity, desire, disappointment, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, optimism, pride, realization, relief, remorse, sadness, surprise, neutral

4.6 Dataset Summary

Dataset	Task	Size	Classes	Source
FER-2013	Face emotion recognition	35,887 images	7 emotions	Kaggle
GoEmotions	Text sentiment analysis	58,000+ comments	28 emotions	HuggingFace Hub

5. Progress Assessment 1 & 2 (Week 6 & 9)

5.1 Progress Assessment 1 – Week 6

Sr. No.	Task / Milestone	Status	Remarks
1	Problem definition and scope finalization	Completed	Defined multimodal emotion detection objectives
2	Dataset acquisition – FER-2013	Completed	Downloaded from Kaggle, organized by class folder
3	Dataset acquisition – GoEmotions	Completed	Downloaded from HuggingFace Hub as CSV splits
4	Data preprocessing pipeline (face)	Completed	Image normalization, augmentation pipeline built
5	Data preprocessing pipeline (text)	Completed	Tokenization and multi-hot label encoding implemented
6	CNN architecture design (EmotionCNN)	Completed	3 conv blocks + FC layers implemented in PyTorch
7	Face model training (50 epochs)	Completed	65% test accuracy achieved on FER-2013
8	Face model evaluation	Completed	Per-class F1 scores generated, confusion matrix plotted
9	Backend Flask skeleton	In Progress	App.py, config.py, and route stubs created
10	DistilBERT fine-tuning	Planned	Not started at Week 6

5.2 Progress Assessment 2 – Week 9

Sr. No.	Task / Milestone	Status	Remarks
1	DistilBERT fine-tuning on GoEmotions	Completed	42.78% subset accuracy, best model saved at epoch 7
2	Text model evaluation (28 classes)	Completed	Per-emotion F1 scores generated; amusement 80% F1
3	Flask backend API (face + text routes)	Completed	Blueprint-based routing, input validation, error handling
4	FaceEmotionService (CNN inference)	Completed	Base64 decode, Haar Cascade, CNN prediction pipeline
5	TextSentimentService (DistilBERT inference)	Completed	Tokenize, forward pass, threshold, sentiment determination
6	React frontend – FaceAnalysis component	Completed	Webcam capture, API call, result display
7	React frontend – TextAnalysis component	Completed	Text input, example buttons, character counter
8	EmotionResult component (charts)	Completed	Recharts bar chart with sorted emotion probabilities

Sr. No.	Task / Milestone	Status	Remarks
9	End-to-end integration testing	Completed	Both modalities tested in live environment
10	Project documentation	In Progress	Report being finalized

6. Implementation and Output

6.1 System Architecture

EmotionAI follows a three-tier architecture: a React SPA (presentation), a Flask REST API (application logic), and PyTorch/HuggingFace ML inference services (data/ML tier). All communication occurs over HTTP using JSON payloads.

Tier	Component	Technology	Responsibility
Presentation	React SPA	React + Vite	User interaction, webcam capture, text input, results display
Application	Flask REST API	Python Flask	Request routing, input validation, service orchestration
ML Inference	Emotion Services	PyTorch + HuggingFace	Model loading, preprocessing, inference, JSON response
Storage	Saved Checkpoints	.pth files	Persisted model weights for CNN and DistilBERT

6.2 Backend Implementation

backend/app.py – Flask Application Entry Point

Creates the Flask app, enables CORS so the React frontend (port 5173) can communicate with the backend (port 5000), registers face_bp and text_bp blueprints, and exposes a /api/health endpoint for connectivity checking.

backend/config.py – Central Configuration

Single source of truth for all configuration: model checkpoint paths (face_emotion_model.pth and text_emotion_model.pth), emotion label lists (7 for face, 28 for text), image size constant (48×48), and maximum token length (128). All other modules import from here.

backend/routes/face_routes.py – Face Prediction Endpoint

Flask Blueprint handling POST /api/face/predict. Accepts a JSON body with a base64-encoded image, validates input, calls FaceEmotionService.predict(), and returns emotion scores as JSON. Returns HTTP 500 on error.

backend/routes/text_routes.py – Text Prediction Endpoint

Flask Blueprint handling POST /api/text/predict. Accepts a JSON body with a text string, validates non-empty input, calls TextSentimentService.predict(), and returns multi-label emotion scores and overall sentiment as JSON.

backend/services/face_emotion_service.py

FaceEmotionService class with three core methods:

- _load_model(): Loads EmotionCNN architecture, reads .pth checkpoint, sets model to eval() mode.
- preprocess_image(): Decodes base64, runs Haar Cascade detection, crops face region, resizes to 48×48, normalizes pixels, returns PyTorch tensor.
- predict(): Runs inference in no_grad context, applies softmax, returns dict of emotion→probability and top prediction.

backend/services/text_sentiment_service.py

TextSentimentService class:

- _load_model(): Loads TextEmotionModel, reads .pth checkpoint, initializes DistilBertTokenizer.
- predict(): Tokenizes text, runs forward pass, applies sigmoid, applies 0.3 threshold, determines sentiment (positive/negative/neutral), returns structured JSON response.

6.3 ML Model Implementation

ml_models/face_model/model.py – EmotionCNN

Defines the EmotionCNN class (nn.Module). Three sequential convolutional blocks each containing two Conv2d layers, BatchNorm2d, ReLU activation, MaxPool2d, and Dropout. Followed by Flatten, FC(4608→256), FC(256→128), and output FC(128→7). The forward() method defines the complete computation graph.

ml_models/face_model/data_loader.py – Data Pipeline

get_transforms(is_train): For training — random horizontal flip, random rotation (±10°), color jitter, resize 48×48, grayscale, normalize. For testing — resize, grayscale, normalize only. get_dataloaders(batch_size): Uses PyTorch ImageFolder (expects images in class-named subdirectories), returns DataLoader objects.

ml_models/face_model/train.py – Training Script

Calculates class weights (inverse frequency) to handle the disgust/happy imbalance. Runs 50 training epochs with Adam optimizer and ReduceLROnPlateau scheduler. Each epoch: forward pass → CrossEntropyLoss → loss.backward() → optimizer.step(). Saves checkpoint on validation accuracy improvement.

ml_models/text_model/model.py – TextEmotionModel

Wraps pretrained DistilBertModel with a custom classifier head: Linear(768→256) + ReLU + BatchNorm + Dropout(0.3) → Linear(256→128) + ReLU + Dropout(0.3) → Linear(128→28) + Sigmoid. The [CLS] token (first token of DistilBERT output) is extracted as the sentence representation.

ml_models/text_model/tokenizer_utils.py – GoEmotionsDataset

GoEmotionsDataset class (torch.utils.data.Dataset): __getitem__ tokenizes text to input_ids + attention_mask (max 128 tokens, padding/truncation), converts label list to 28-dimensional multi-hot tensor. get_dataloaders() wraps train/val/test CSVs into DataLoader objects.

ml_models/text_model/train.py – DistilBERT Training

Uses BCEWithLogitsLoss for multi-label training, AdamW optimizer with weight decay, StepLR scheduler, and gradient clipping (max_norm=1.0) to prevent exploding gradients. Supports checkpoint resumption via RESUME=True flag. Saves best model on validation loss improvement.

6.4 Frontend Implementation

frontend/src/App.jsx – Root Component

Manages global tab state ('face' or 'text'). Renders application header with EmotionAI branding, tab switcher buttons, and conditionally renders FaceAnalysis or TextAnalysis component.

frontend/src/components/FaceAnalysis.jsx

Uses react-webcam for camera access. On 'Capture & Analyze' click: calls getScreenshot() to get JPEG frame, encodes as base64, calls analyzeFace() API. Manages state: camera on/off, loading, result, snapshot preview, errors.

frontend/src/components/TextAnalysis.jsx

Provides textarea with real-time character counter, example sentence buttons for quick testing, and 'Analyze' button. Calls analyzeText() API and passes result to EmotionResult. Manages state: text input, loading, result, errors.

frontend/src/components/EmotionResult.jsx

Reusable result display component shared by both analysis modes. Shows: detected emotion emoji, emotion label, confidence percentage, sentiment badge (positive/negative/neutral), and a sorted Recharts bar chart of all emotion probabilities.

frontend/src/api/emotionApi.js

Centralizes all API calls. analyzeFace(base64Image): POST to /api/face/predict. analyzeText(text): POST to /api/text/predict. checkHealth(): GET /api/health. Uses axios pointing to http://127.0.0.1:5000.

6.5 Data Flow

Face Analysis Flow:

1. User clicks 'Start Camera' → browser requests webcam permission.
2. User clicks 'Capture & Analyze' → react-webcam captures JPEG frame.
3. Frame encoded as base64 → sent via axios POST to /api/face/predict.
4. Flask → FaceEmotionService.predict() called.
5. OpenCV Haar Cascade detects face → crops and resizes to 48×48 pixels.
6. Image normalized → converted to PyTorch tensor.
7. CNN forward pass → 7 logits → softmax → probabilities.
8. Top emotion + all scores returned as JSON → displayed in UI.

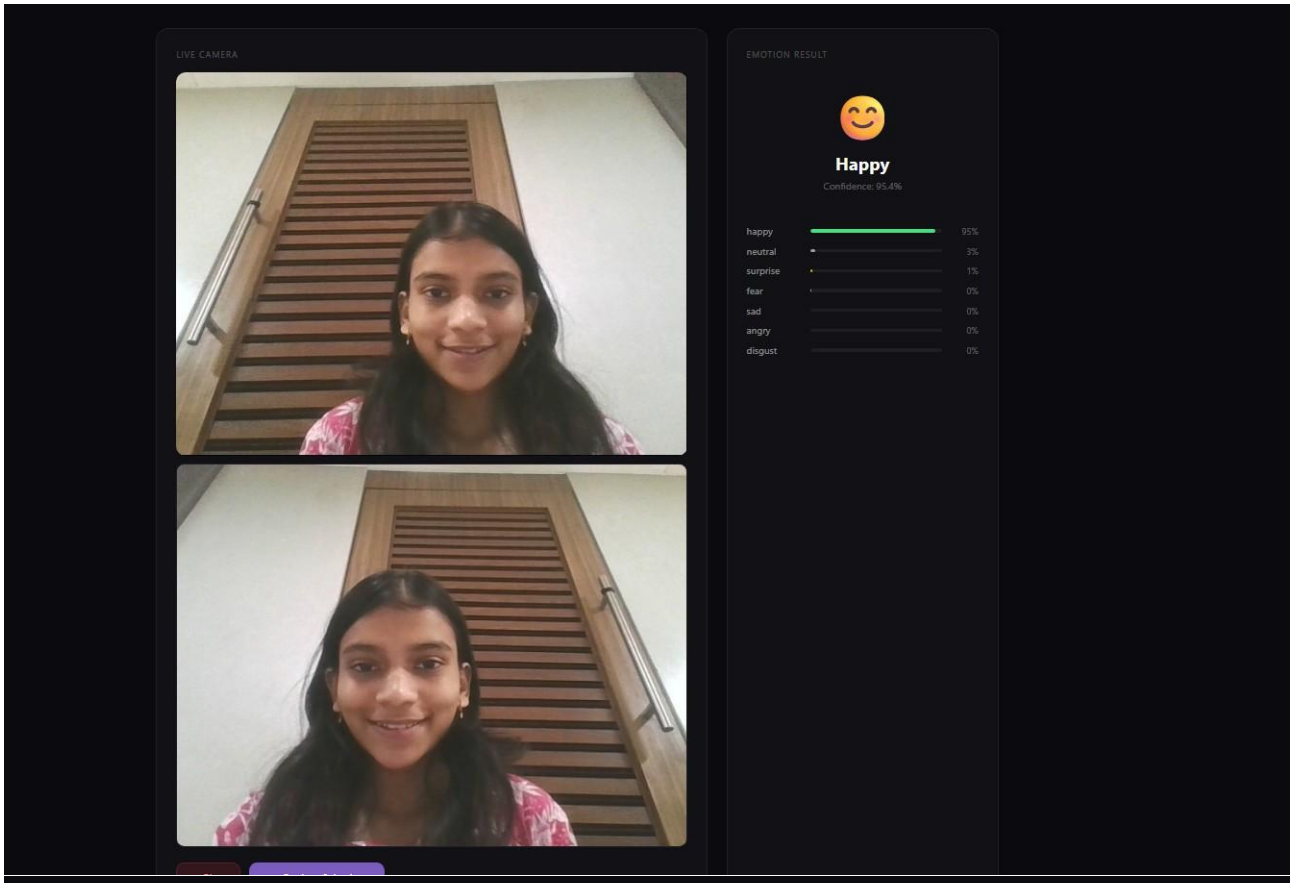
Text Analysis Flow:

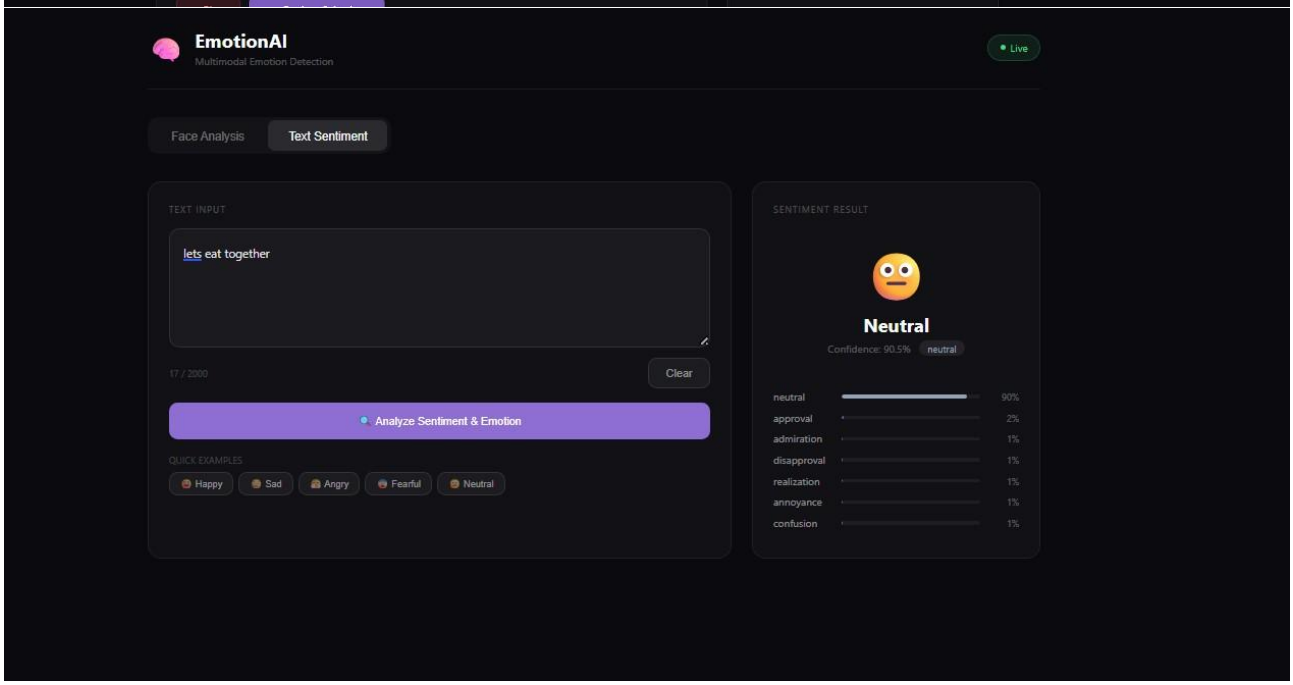
9. User types text → clicks 'Analyze Sentiment & Emotion'.
10. Text sent via POST to /api/text/predict.
11. Flask → TextSentimentService.predict().
12. DistilBertTokenizer encodes text → input_ids + attention_mask.
13. DistilBERT forward pass → 28 logits → sigmoid → probabilities.
14. Threshold 0.3 applied → active emotions identified.
15. Sentiment determined from top emotion.
16. Results JSON returned → rendered with bars, badges, emoji in UI.

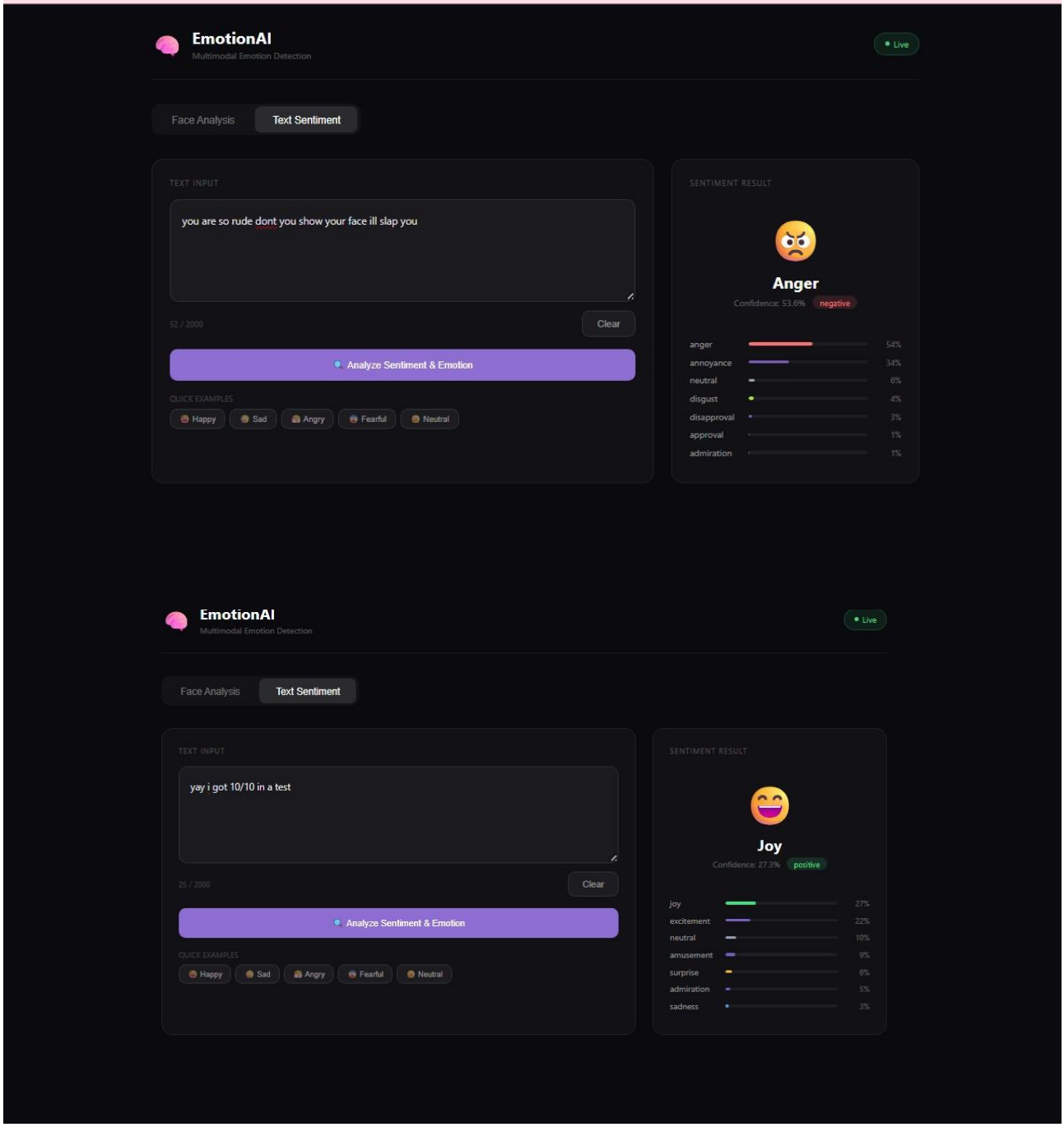
6.6 Project Folder Structure

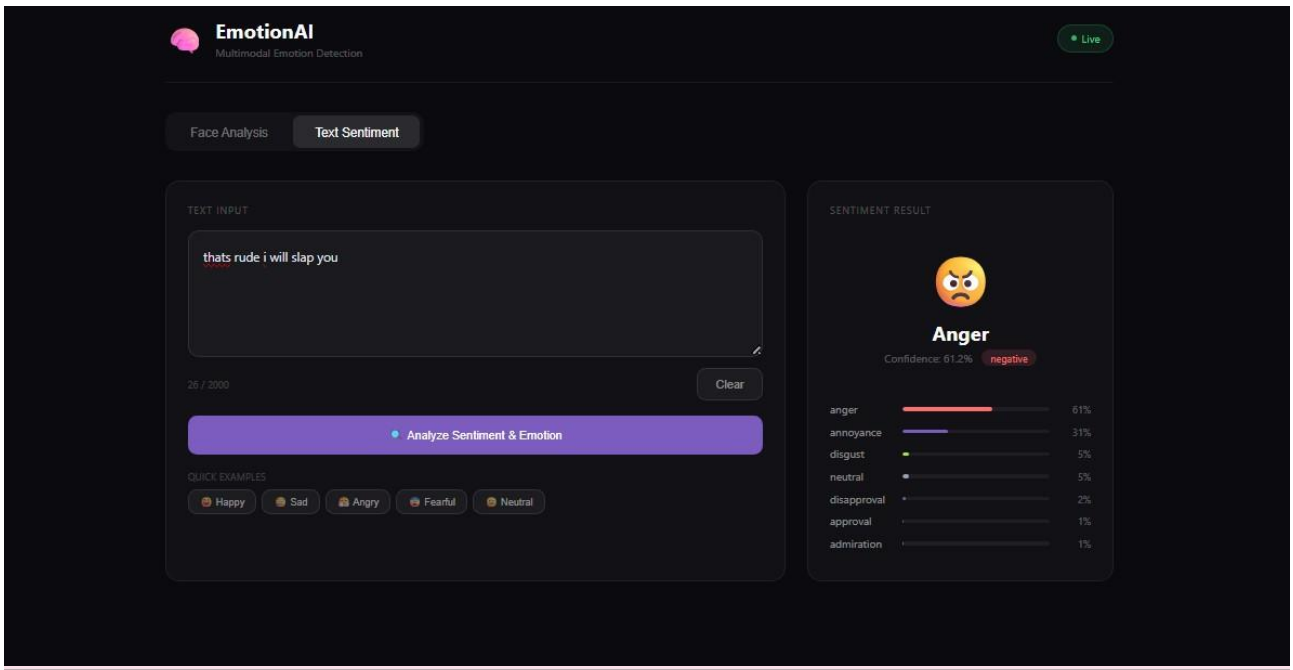
Path	Description
backend/app.py	Flask app entry point — creates server, registers blueprints
backend/config.py	Central config: model paths, label lists, constants
backend/routes/face_routes.py	/api/face/predict endpoint (Blueprint)
backend/routes/text_routes.py	/api/text/predict endpoint (Blueprint)
backend/services/face_emotion_service.py	CNN inference: preprocessing + prediction
backend/services/text_sentiment_service.py	DistilBERT inference: tokenization + prediction
ml_models/face_model/model.py	EmotionCNN class definition
ml_models/face_model/data_loader.py	Image augmentation pipeline, DataLoader creation
ml_models/face_model/train.py	Face CNN training script (50 epochs)
ml_models/face_model/evaluate.py	Face model test evaluation + per-class report
ml_models/text_model/model.py	TextEmotionModel (DistilBERT + classifier head)
ml_models/text_model/tokenizer_utils.py	GoEmotionsDataset class + DataLoader creation
ml_models/text_model/train.py	DistilBERT fine-tuning training script
ml_models/text_model/evaluate.py	Text model evaluation across 28 emotion classes

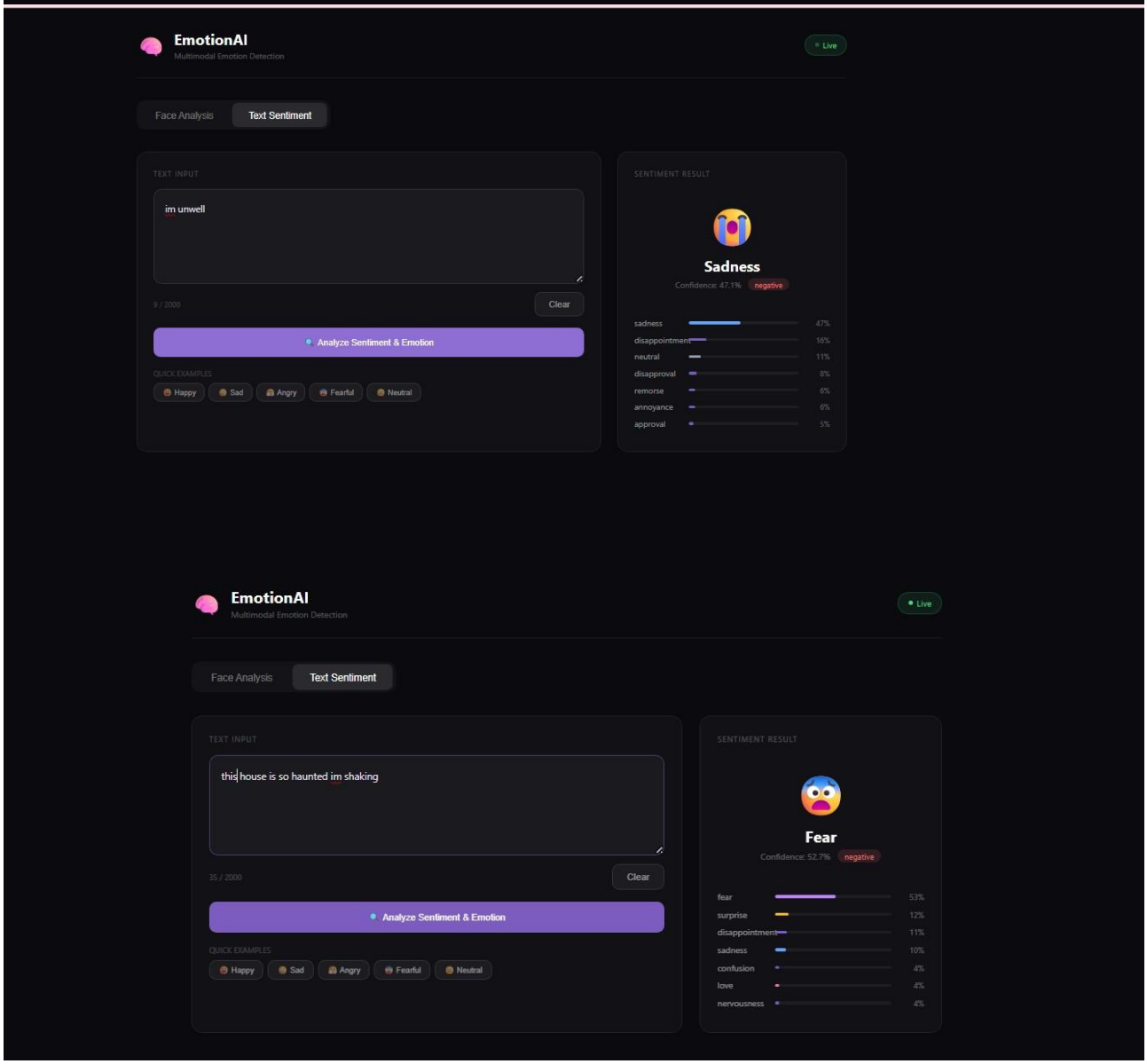
Path	Description
ml_models/saved_models/face_emotion_model.pth	Saved CNN checkpoint (best validation accuracy)
ml_models/saved_models/text_emotion_model.pth	Saved DistilBERT checkpoint (best validation loss)
ml_models/datasets/raw/fer2013/	FER-2013 images organized by class folder
ml_models/datasets/processed/text/	GoEmotions CSV files (train/val/test splits)
frontend/src/App.jsx	Root React component with tab management
frontend/src/api/emotionApi.js	Centralized axios-based API call functions
frontend/src/components/FaceAnalysis.jsx	Webcam capture and face analysis component
frontend/src/components/TextAnalysis.jsx	Text input and sentiment analysis component
frontend/src/components/EmotionResult.jsx	Reusable emotion result display with Recharts
frontend/src/styles/globals.css	Global dark-theme CSS styles
emotion_env/	Python virtual environment


LIVE CAMERA
EMOTION RESULT
Happy
Confidence: 95.4%
happy 95%
neutral 3%
surprise 1%
fear 0%
sad 0%
angry 0%
disgust 0%


EmotionAI
Multimodal Emotion Detection
Live
Face Analysis
Text Sentiment
TEXT INPUT
lets eat together
17 / 2000
Clear
Analyze Sentiment & Emotion
QUICK EXAMPLES
Happy
Sad
Angry
Fearful
Neutral
SENTIMENT RESULT
Neutral
Confidence: 90.5%
neutral
neutral 90%
approval 2%
admiration 1%
disapproval 1%
realization 1%
annoyance 1%
confusion 1%


EmotionAI
Multimodal Emotion Detection
Live
Face Analysis
Text Sentiment
TEXT INPUT
you are so rude dont you show your face ill slap you
52 / 2000
Clear
Analyze Sentiment & Emotion
QUICK EXAMPLES
Happy
Sad
Angry
Fearful
Neutral
SENTIMENT RESULT
Anger
Confidence: 53.6%
negative
anger 54%
annoyance 34%
neutral 6%
disgust 4%
disapproval 3%
approval 1%
admiration 1%
EmotionAI
Multimodal Emotion Detection
Live
Face Analysis
Text Sentiment
TEXT INPUT
yay i got 10/10 in a test
25 / 2000
Clear
Analyze Sentiment & Emotion
QUICK EXAMPLES
Happy
Sad
Angry
Fearful
Neutral
SENTIMENT RESULT
Joy
Confidence: 27.3%
positive
joy 27%
excitement 22%
neutral 10%
amusement 9%
surprise 6%
admiration 5%
sadness 3%


EmotionAI
Multimodal Emotion Detection
Live
Face Analysis
Text Sentiment
TEXT INPUT
thats rude i will slap you
26 / 2000
Clear
Analyze Sentiment & Emotion
QUICK EXAMPLES
Happy
Sad
Angry
Fearful
Neutral
SENTIMENT RESULT
Anger
Confidence: 61.2%
negative
anger 61%
annoyance 31%
disgust 5%
neutral 5%
disapproval 2%
approval 1%
admiration 1%


EmotionAI
Multimodal Emotion Detection
Live
Face Analysis
Text Sentiment
TEXT INPUT
im unwell
9 / 2000
Clear
Analyze Sentiment & Emotion
QUICK EXAMPLES
Happy
Sad
Angry
Fearful
Neutral
SENTIMENT RESULT
Sadness
Confidence: 47.1%
negative
sadness 47%
disappointment 16%
neutral 11%
disapproval 8%
remorse 6%
annoyance 6%
approval 5%

EmotionAI
Multimodal Emotion Detection
Live
Face Analysis
Text Sentiment
TEXT INPUT
this house is so haunted im shaking
35 / 2000
Clear
Analyze Sentiment & Emotion
QUICK EXAMPLES
Happy
Sad
Angry
Fearful
Neutral
SENTIMENT RESULT
Fear
Confidence: 52.7%
negative
fear 53%
surprise 12%
disappointment 11%
sadness 10%
confusion 4%
love 4%
nervousness 4%

7. Observations and Learnings

7.1 Technical Observations

- Dataset quality has a major impact on model performance. FER-2013 contains noisy and ambiguous labels, contributing to the ~65% accuracy ceiling for the CNN.
- Class imbalance in FER-2013 (disgust: 111 vs. happy: 7,215 training samples) significantly impacts per-class F1 scores. Even with weighted CrossEntropyLoss, rare classes remain difficult to learn.
- Transfer learning with pretrained DistilBERT dramatically reduced training time compared to training from scratch. Only 10 fine-tuning epochs were required to achieve competitive 28-class multi-label accuracy.
- Subset accuracy (42.78%) for 28-class multi-label classification is a stringent metric — all 28 labels must match exactly. Individual per-class F1 scores (e.g., amusement: 80%, admiration: 68%) better reflect model quality.
- The Haar Cascade face detector is efficient for real-time use but degrades under poor lighting, extreme angles, or partial occlusion. Deep-learning detectors (MTCNN, MediaPipe) would be more robust.
- CORS configuration was essential to allow the React dev server (port 5173) to communicate with the Flask backend (port 5000).
- The 0.3 sigmoid threshold for text emotion inference required empirical tuning; lower thresholds increased false positives while higher thresholds missed valid secondary emotions.

7.2 Key Learning Outcomes

- End-to-End ML Systems: Built a complete pipeline from raw dataset acquisition through training, evaluation, API serving, and frontend integration.
- Transfer Learning: Learned to fine-tune large pretrained transformer models (DistilBERT) for domain-specific multi-label classification.
- Multimodal AI Architecture: Designed a system that integrates two independent AI modalities (vision and language) in a unified application.
- REST API Design: Developed skills in Flask Blueprint-based routing, input validation, CORS handling, and structured JSON error responses.
- Full-Stack Development: Practiced connecting a React SPA frontend to a Python ML backend via HTTP, managing asynchronous state in React.
- Model Evaluation Metrics: Deepened understanding of precision, recall, F1-score for single-label classification and subset accuracy for multi-label tasks.
- Class Imbalance Handling: Learned to calculate and apply inverse-frequency class weights in loss functions to address severe dataset imbalances.

8. Future Scope

8.1 Model Improvements

- Upgrade the face model from a custom CNN to ResNet-50, EfficientNet, or Vision Transformer (ViT) architectures which have achieved 70–75% accuracy on FER-2013.
- Apply Mixup and CutMix augmentation strategies to artificially increase training diversity.
- Replace Haar Cascade with a deep learning-based face detector (MTCNN, MediaPipe, or RetinaFace) for better robustness under varied conditions.
- Experiment with semi-supervised learning to leverage unlabeled facial expression data.
- Apply model distillation to reduce model size for mobile/edge deployment.

8.2 New Modalities

- Audio / Speech Emotion Recognition: Integrate a speech emotion model using MFCCs with RNNs or audio transformers.
- Video-Based Temporal Analysis: Process video streams to track emotional state transitions over time using LSTM/GRU on sequential CNN features.
- Physiological Signals: Explore wearable sensor data (heart rate, galvanic skin response) for multi-modal physiological emotion sensing.

8.3 Application Enhancements

- Implement real-time continuous face analysis (video stream mode) rather than single-frame capture.
- Add emotion history tracking with time-series visualization using a database (SQLite or PostgreSQL).
- Build a mobile application using React Native sharing the same Flask backend API.
- Implement multi-face detection and emotion tracking for group scenarios such as classrooms or meetings.
- Add multilingual text emotion analysis using multilingual transformers (mBERT, XLM-R).

8.4 Deployment

- Containerize the application with Docker for environment consistency.
- Deploy to cloud platforms (AWS / GCP / Azure) with gunicorn as the production WSGI server.
- Implement API authentication, rate limiting, and logging for production use.
- Explore ONNX export and model quantization for faster inference at reduced memory.

9. Conclusion

EmotionAI successfully delivers a multimodal emotion detection system that integrates facial expression recognition and text sentiment analysis into a cohesive, production-ready web application. The project demonstrates that sophisticated AI capabilities can be made accessible through thoughtful software engineering and clean user interface design.

The key achievements of this project are:

- A custom CNN trained on FER-2013 achieved 65% accuracy across 7 emotion classes, with strong performance on well-represented classes (happy: F1 0.86, surprise: F1 0.76).
- A fine-tuned DistilBERT model achieved 42.78% subset accuracy across 28 GoEmotions categories, with best-class F1 scores of 80% (amusement) and 68% (admiration).
- A production-ready Flask REST API exposes both models via clean Blueprint-based endpoints with proper input validation and error handling.
- A modern React SPA provides real-time webcam capture, text input, and visual emotion probability displays through an intuitive dark-themed interface.

The modular three-tier architecture ensures each component — ML training, backend API, and frontend — can evolve independently. This makes EmotionAI a strong foundation for future enhancements including additional modalities, improved architectures, multi-user tracking, and cloud deployment. The project provided invaluable hands-on experience in end-to-end machine learning systems development, full-stack web engineering, and practical deployment considerations.

10. References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org
2. Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.
3. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT. arXiv:1910.01108.
4. Demszky, D., et al. (2020). GoEmotions: A Dataset of Fine-Grained Emotions. ACL 2020. Google Research.
5. Goodfellow, I., et al. (2013). Challenges in Representation Learning: FER-2013. arXiv:1307.0414.
6. PyTorch Documentation. (2024). https://pytorch.org/docs/
7. HuggingFace Transformers Documentation. (2024). https://huggingface.co/docs/transformers
8. OpenCV Documentation. (2024). https://docs.opencv.org/
9. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.
10. Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. CVPR 2001.
11. Flask Documentation. (2024). https://flask.palletsprojects.com/
12. React Documentation. (2024). https://react.dev/
13. Loshchilov, I., & Hutter, F. (2017). Decoupled Weight Decay Regularization (AdamW). arXiv:1711.05101.
14. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT. NAACL-HLT 2019.
15. Kaggle FER-2013 Dataset. https://www.kaggle.com/datasets/msambare/fer2013